

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Rosario

Tecnicatura Universitaria en Programación a Distancia

Materia: Programación 1

Trabajo Práctico Integrador: Gestión de Datos de Países en Python

Año: 2026

INTEGRANTES DEL GRUPO

- Walter Matias Kurtz (Comisión 15)
- Rolando Alanis (Comisión 13)

1. INTRODUCCIÓN Y PLANTEO DEL PROBLEMA

El presente proyecto consiste en el diseño e implementación de una aplicación de consola en Python destinada a la gestión, análisis y persistencia de datos demográficos y geográficos de diversos países.

El problema principal radica en la necesidad de procesar un archivo fuente en formato plano (países.csv) que actúa como persistencia física, transformando dichos registros analógicos en estructuras dinámicas en memoria para permitir operaciones CRUD (Alta, Baja, Modificación y Consulta), filtrados por criterios específicos, ordenamientos algorítmicos complejos y generación de indicadores estadísticos. Todo esto bajo un entorno robusto que garantice la integridad de los datos frente a potenciales fallas en el ingreso por parte del usuario.

2. ESTRUCTURA DE DATOS UTILIZADA

Para la manipulación eficiente de la información en memoria orientada al paradigma estructurado, se optó por una **lista de diccionarios**.

- **La Lista Principal (países):** Funciona como la colección contenedora global indexada numéricamente. Permite el recorrido secuencial iterativo mediante estructuras de control bucle (for), la adición dinámica de nuevos registros y la eliminación física de elementos de forma directa.
- **Los Diccionarios Internos:** Cada país está representado por un diccionario independiente que contiene un mapeo de claves (Key) y valores (Value). Las claves estandarizadas son: nombre, continente, poblacion y superficie.

Justificación técnica: El uso de diccionarios optimiza la legibilidad del código y el acceso a los atributos, ya que permite referenciar las propiedades de un elemento mediante etiquetas semánticas (ej. pais["poblacion"]) en lugar de depender de índices numéricos fijos de una tupla o lista multidimensional, lo que prevendría errores ante eventuales reestructuraciones del archivo fuente.

3. ARQUITECTURA Y MODULARIZACIÓN

El software se diseñó bajo el principio de modularización, dividiendo la lógica de negocio y la interfaz de usuario en archivos separados para asegurar la mantenibilidad y legibilidad del código:

1. **funciones.py (Capa de Lógica y Algoritmia):** Concentra la totalidad de las funciones del sistema. No gestiona el flujo principal del menú, sino que recibe las estructuras de datos como parámetros, ejecuta los algoritmos correspondientes y retorna resultados o manipula las listas por referencia.
2. **main.py (Capa de Interfaz y Control):** Actúa como el punto de entrada exclusivo de la aplicación. Centraliza la llamada inicial de carga del archivo externo y encapsula el Menú Interactivo de opciones mediante un bucle de

control condicional (while True) y una estructura de selección múltiple (if-elif-else).

4. DETALLE TÉCNICO DE LAS FUNCIONALIDADES DESARROLLADAS

El desarrollo de las funciones del sistema se dividió de manera equitativa entre los integrantes del equipo:

Bloque de Funciones A (Desarrollado por Rolando Alanis):

- **Lectura e Inicialización (cargar_paises):** Implementa la apertura del archivo físico paises.csv mediante la función nativa open() en modo de lectura y utiliza csv.DictReader para estructurar automáticamente las filas en diccionarios en memoria.
- **Visualización General (mostrar_paises):** Recorre de manera secuencial la lista principal imprimiendo formateadamente los datos en la terminal.
- **Alta de Registros (agregar_pais):** Captura las variables por consola, valida la completitud de los campos y añade el nuevo objeto diccionario a la lista global.
- **Modificación de Registros (actualizar_pais):** Localiza secuencialmente un registro mediante su clave nominal y sobrescribe dinámicamente los valores analizados (población o superficie) según los requerimientos del usuario.
- **Módulo Analítico (estadísticas):** Obtiene los valores extremos empleando max() y min() combinados con expresiones lambda para evaluar propiedades específicas. Ejecuta sumatorias aritméticas para determinar promedios generales y utiliza un diccionario acumulador dinámico para registrar la distribución exacta de países por continente.

Bloque de Funciones B (Desarrollado por Walter Matias Kurtz):

- **Búsqueda Selectiva (buscar_pais):** Implementa un motor de búsqueda secuencial que contrasta el criterio del usuario con la clave nominal de los elementos almacenados.
- **Extracción por Criterio (filtrar_por_continente):** Segmenta la base de datos acumulando y mostrando en pantalla únicamente aquellos registros que coinciden exactamente con la región geográfica consultada, gestionado por una variable lógica de control (bandera).
- **Baja de Registros (eliminar_pais):** Ejecuta la remoción física del diccionario correspondiente dentro de la lista utilizando el método analítico .remove().
- **Persistencia de Salida (guardar_paises):** Garantiza que los cambios realizados en memoria se vuelquen de forma definitiva al disco. Abre el archivo en modo escritura ('w'), define las cabeceras requeridas e instancia csv.DictWriter para reescribir ordenadamente el dataset original.

5. EXPLICACIÓN DETALLADA DEL ALGORITMO DE ORDENAMIENTO

Para cumplir con las pautas de diseño lógico elemental exigidas por la cátedra, no se utilizaron métodos automáticos de ordenamiento provistos por el lenguaje (como `.sort()` o `sorted()`). En su lugar, se programó manualmente el algoritmo clásico de **Ordenamiento por Burbuja (Bubble Sort)**.

La función `ordenar_paises` utiliza dos bucles de repetición anidados:

1. **Bucle Externo:** Controla las pasadas generales necesarias sobre la estructura. En el peor de los casos, se requieren tantas pasadas como elementos tenga la lista menos uno ($N-1$).
2. **Bucle Interno:** Realiza el recorrido comparativo lineal par por par. Compara el atributo numérico `poblacion` del índice actual `j` con el del índice contiguo `j + 1`.

Si el valor del elemento izquierdo es estrictamente mayor que el del elemento derecho, significa que se encuentran desordenados según el criterio de menor a mayor. En ese instante se realiza un **intercambio (swap)**. Para evitar la pérdida de datos en memoria durante la reasignación, se almacena temporalmente el diccionario completo del índice `j` dentro de una variable auxiliar denominada `aux`. Una vez resguardado, se procede a reubicar las posiciones físicas dentro de la lista. Este proceso se repite iterativamente hasta que el set de datos queda homogéneamente ordenado.

6. ROBUSTEZ Y MANEJO DE ERRORES (TRY/EXCEPT)

La estabilidad ante fallos en tiempo de ejecución (Runtime Errors) es un pilar central de la aplicación. El programa previene activamente el colapso de la consola (crash) mediante el uso estratégico de bloques de control de excepciones `try-except`:

1. **Robustez en la Carga de Archivos:** Durante la conversión sintáctica de tipos de datos al leer el CSV, si una fila posee un campo numérico corrupto, vacío o con caracteres alfabéticos, el método `int()` arrojaría un error de valor (`ValueError`). El bloque `try-except` captura este fallo, evita la interrupción abrupta del sistema, reporta el registro anómalo por consola y permite que el bucle continúe procesando los países restantes de la base de datos.
2. **Robustez en la Interfaz de Usuario:** En los módulos de captura de datos (Menú Principal, Altas y Modificaciones), se implementaron estructuras `try-except` para blindar los ingresos por teclado. Si el sistema solicita un valor numérico entero y el usuario ingresa texto de forma errónea, la excepción es capturada de inmediato, desplegando un mensaje de advertencia e impidiendo el ingreso de datos inconsistentes en las estructuras de memoria.

7. CONCLUSIÓN

El desarrollo de este Trabajo Práctico Integrador permitió consolidar de manera práctica los fundamentos de la programación estructurada y la manipulación de flujos de datos bajo el entorno Python. La modularización demostró ser una estrategia indispensable para organizar el trabajo en equipo de forma ordenada y sin solapamiento de código.

Como conclusión técnica, se destaca que la combinación de listas y diccionarios dota al lenguaje de una flexibilidad sumamente potente para simular bases de datos relacionales en proyectos de pequeña y mediana escala, logrando un software interactivo, eficiente en su algoritmia elemental y altamente tolerante a fallos de operación.